

Đề 01

Lập 2 chương trình khác nhau trên ngôn ngữ lập trình tùy chọn C++/Java/C# với yêu cầu sau:

Lập chương trình CT1 (có hàm main):

1. Viết 1 hàm gồm 1 vòng lặp tạo liên tục 1 số nguyên không âm (số nguyên 4byte), nếu số được tạo chia hết cho 2021 thì thoát khỏi vòng lặp, trái lại thì ghi số nguyên vào file `dulieu.dat` (file mở ở mode ghi đè nếu file đã tồn tại), khi ghi được thì đóng file.
2. Đặt hàm vào 1 luồng, kích hoạt luồng trong hàm `main()`.

Lập chương trình CT2 (có hàm main):

3. Viết 1 hàm có 1 vòng lặp vô hạn đọc số nguyên 4 byte ở file `dulieu.dat`. Đọc được số nguyên thì đóng file. Hiển thị giá trị đọc được. Nếu số nguyên đọc được chia hết cho 2021 thì thoát vòng lặp.
4. Đặt hàm vào 1 luồng, kích hoạt luồng trong hàm `main()`.

Chạy cả CT1 và CT2 đồng thời để thấy hiệu ứng trao đổi dữ liệu.

Đề 02

Lập 1 chương trình trên ngôn ngữ lập trình tùy chọn C++/Java/C# gồm 2 tác vụ (Task1, Task2) hoạt động ở chế độ không đồng bộ như sau:

Task1:

1. Viết 1 hàm gồm 1 vòng lặp nhập 1 chuỗi ký tự `st` từ bàn phím (`st` là một biến tổng thể). Loại bỏ các dấu cách ở đầu và cuối chuỗi `st`. Nếu chuỗi `st` được nhập và xử lý cắt dấu cách là `"bye"` thì thoát khỏi vòng lặp.
2. Thiết lập Task1 bằng cách đặt hàm trên vào 1 timer hoặc 1 luồng, kích hoạt timer/luồng trong hàm `main()`.

Task2:

3. Viết 1 hàm có 1 vòng lặp vô hạn hiển thị biến chuỗi ký tự `st` (xem câu 1). Nếu `st` là `"bye"` thì thoát vòng lặp. Bổ sung lệnh gọi hàm trong hàm `main()`.
4. Thiết lập Task2 bằng cách đặt hàm trên vào 1 timer hoặc 1 luồng, kích hoạt timer/luồng trong hàm `main()`. Dịch và chạy chương trình.

Đề 03

Lập 1 chương trình trên ngôn ngữ lập trình tùy chọn C++/Java/C# gồm 2 Timer (Timer1, Timer2), hoạt động như sau:

Timer1:

1. Viết 1 hàm gồm 1 vòng lặp nhập 1 kí tự từ bàn phím, lưu vào 1 biến tổng thể **c**, hiển thị dạng hexa của kí tự. Nếu kí tự là '*' thì thoát khỏi vòng lặp.
2. Đặt hàm vào 1 timer (Timer1) cứ 15ms (1ms=1/1000 giây) thực hiện 1 lần, kích hoạt Timer1 trong hàm main().

Timer2:

3. Viết 1 hàm có 1 vòng lặp phát ra tiếng beep, nếu biến tổng thể **c** là '*' thì thoát vòng lặp (xem câu 1). Bổ sung lệnh gọi hàm trong hàm main.
 4. Đặt hàm vào 1 timer (Timer2) cứ 7ms thực hiện 1 lần, kích hoạt Timer2 trong hàm main().
- Dịch và chạy chương trình.

Đề 04

Lập 1 chương trình trên ngôn ngữ lập trình tùy chọn C++/Java/C# gồm 2 tác vụ (Task1, Task2) hoạt động ở chế độ không đồng bộ như sau:

Task1:

1. Viết 1 hàm gồm 1 vòng lặp sinh số nguyên ngẫu nhiên cho đến khi gặp số lớn hơn 10000 và chia hết cho 2021 thì dừng vòng lặp. Viết hàm main() gọi hàm, dịch và chạy chương trình.
2. Thiết lập **Task1** bằng cách đặt hàm trên vào 1 timer chu kỳ 1 mili giây hoặc 1 luồng, kích hoạt timer/luồng trong hàm main().

Task2:

3. Viết 1 hàm có 1 vòng lặp chỉ dừng lặp khi vòng lặp của Task1 dừng lại. Vòng lặp này luôn hiển thị số lớn nhất và số lớn nhì (số lớn nhất thứ hai, nếu có) được **Task1** tạo ra tại mọi thời điểm chạy. Bổ sung lệnh gọi hàm trong hàm main().
4. Thiết lập **Task2** bằng cách đặt hàm trên vào 1 timer hoặc 1 luồng, kích hoạt luồng trong hàm main(). Thêm lệnh phát ra tiếng beep khi hiển thị. Dịch và chạy chương trình.

Đề 05

Lập 2 chương trình khác nhau trên ngôn ngữ lập trình tùy chọn **C++/Java/C#** gồm **CT1**(client), **CT2** (server) sử dụng Socket API, giao thức TCPIP truyền nhận dữ liệu tại IP="127.0.0.1" (localhost), port=5000 và hoạt động như sau:

CT1 (có hàm main):

1. Viết 1 hàm gồm 1 vòng lặp tạo ra số nguyên ngẫu nhiên **k**, vòng lặp dừng khi số nguyên sinh ra lớn hơn 10000 và chia hết cho 2022. Viết hàm main() gọi hàm.
2. Bổ sung lệnh truyền giá trị số nguyên sinh ra cho **CT2** trong vòng lặp. Dịch chương trình **CT1**.

CT2 (có hàm main):

3. Viết 1 hàm có 1 vòng lặp vô hạn nhận giá trị biến nguyên được gửi đến từ **CT1**. Kiểm tra, nếu **CT2** nhận được giá trị nguyên **k** thì:

Trường hợp 1: Nếu **k > 10000** và chia hết cho 2022 thì thoát khỏi vòng lặp.

Trường hợp 2: Trái lại, hiển thị giá trị lớn nhất được **CT1** sinh ra tính tại thời điểm đang chạy (giá trị số lớn nhất có thể luôn thay đổi khi **CT1** đang chạy). Viết hàm main(), dịch chương trình **CT2**.

Rồi chạy **CT2** và chạy **CT1** để xem hiệu ứng nhập mảng ở **CT1**, hiển thị mảng ở **CT2**.

4. Đặt hàm ở câu 3 vào 1 timer, bổ sung lệnh kích hoạt timer trong hàm main(). Dịch chương trình.

Biên dịch và chạy lần sau cùng:

+ Đóng tất cả **CT1**, **CT2** đang chạy.

+ Chạy chương trình **CT2** sau đó chạy **CT1** để xem hiệu ứng kết nối, truyền và nhận dữ liệu.

Đề 06

Lập 1 chương trình trên ngôn ngữ lập trình tùy chọn **C++/Java/C#** gồm 2 tác vụ (Task1, Task2) hoạt động ở chế độ không đồng bộ như sau:

Task1:

1. Viết 1 hàm gồm 1 vòng lặp nhập liên tục các số nguyên từ bàn phím, lưu các số nhập được vào một biến cấu trúc **A** kiểu bộ đệm vòng, và vòng lặp dừng lại khi nhập số nguyên < 0.
2. Thiết lập **Task1** bằng cách đặt hàm trên vào 1 luồng, kích hoạt luồng trong hàm main().

Task2:

3. Viết 1 hàm có 1 vòng lặp chỉ dừng lặp khi vòng lặp của Task1 dừng lại. Vòng lặp này luôn hiển thị các số nguyên được lưu trong biến **A**. Bổ sung lệnh gọi hàm trong hàm main().
4. Thiết lập **Task2** bằng cách đặt hàm trên vào 1 luồng, kích hoạt luồng trong hàm main(). Hiển thị trung bình cộng các phần tử của mảng. Dịch và chạy chương trình.

Đề 07

Lập 1 chương trình trên ngôn ngữ lập trình tùy chọn **C++/Java/C#** tạo ra 2 luồng (Thread1, Thread2) hoạt động như sau:

Thread1:

1. Viết 1 hàm gồm mở 1 file chẳng hạn vd.txt ở chế độ text (giả định đây là một file có kích thước rất lớn). Hàm có 1 vòng lặp đọc từng kí tự của file và hiển thị dạng mã hexa của kí tự, vòng lặp dừng khi không đọc tiếp được (đã đọc hết).
2. Đặt hàm trên vào 1 luồng, kích hoạt luồng trong hàm main().

Thread 2:

3. Viết 1 hàm có 1 vòng lặp chỉ dừng lặp khi vòng lặp của Thread1 dừng lại. Vòng lặp này luôn hiển thị các kí tự đã đọc được từ file ở **Thread1**. Bổ sung lệnh gọi hàm trong hàm main().
4. Thiết lập **Thread2** bằng cách đặt hàm trên vào 1 luồng, sửa hàm main() để kích hoạt luồng **Thread2** trước, rồi kích hoạt luồng **Thread1**. Dịch và chạy chương trình.

Đề 08

Lập 2 chương trình khác nhau trên ngôn ngữ lập trình tùy chọn **C++/Java/C#** gồm **CT1**(client), **CT2** (server) sử dụng Socket API, giao thức TCPIP truyền nhận dữ liệu tại IP="127.0.0.1" (localhost), port=5000 và hoạt động như sau:

CT1 (có hàm main):

1. Viết 1 hàm gồm 1 vòng lặp tạo ra số thực ngẫu nhiên **t**, vòng lặp dừng khi số thực **t** sinh ra rơi vào đoạn [0.8,0.82]. Viết hàm main() gọi hàm.
2. Bổ sung lệnh truyền giá trị số thực **t** sinh ra cho **CT2** trong vòng lặp và dịch CT1 (nhưng không chạy)

CT2 (có hàm main):

3. Viết 1 hàm có 1 vòng lặp vô hạn nhận giá trị biến số thực được gửi đến từ **CT1**. Kiểm tra, nếu **CT2** nhận được giá trị số thực **t** thì:

Trường hợp 1: Nếu **t** rơi vào đoạn [0.8,0.82] thì thoát khỏi vòng lặp.

Trường hợp 2: Trái lại, hiển thị giá trị nhỏ nhất được **CT1** sinh ra tính tại thời điểm đang chạy (giá trị số nhỏ nhất có thể luôn thay đổi khi **CT1** đang chạy).

Dịch CT2, chạy **CT2** và chạy **CT1** để xem hiệu ứng.

4. Đặt hàm ở câu 3 vào 1 luồng, bổ sung lệnh kích hoạt luồng trong hàm main(). Dịch chương trình.

Biên dịch và chạy lần sau cùng:

+ Đóng tất cả **CT1**, **CT2** đang chạy.

+ Chạy chương trình **CT2** sau đó chạy **CT1** để xem hiệu ứng kết nối, truyền và nhận dữ liệu.

Đề 09

Lập một chương trình trên ngôn ngữ lập trình tùy chọn C++/Java/C# gồm hai tác vụ (**Task1**, **Task2**) hoạt động ở chế độ không đồng bộ như sau:

Task1:

1. Viết một hàm liên tục nhập các số nguyên từ bàn phím, lưu các số vào một cấu trúc dữ liệu S kiểu danh sách liên kết. Chương trình dừng nhập khi nhập số -1. Viết hàm main() gọi hàm, dịch và chạy chương trình.
2. Thiết lập Task1 bằng cách đặt hàm trên vào một luồng, kích hoạt luồng trong hàm main(). Dịch và chạy lại chương trình.

Task2:

3. Viết một hàm có vòng lặp chỉ dừng khi vòng lặp của Task1 kết thúc. Vòng lặp này luôn hiển thị các số đã lưu trong danh sách liên kết S. Bổ sung lệnh gọi hàm trong main(), dịch và chạy lại chương trình.
4. Thiết lập Task2 bằng cách đặt hàm trên vào một luồng, kích hoạt luồng trong main(). Dịch và chạy chương trình.

Chú ý: Có thể sử dụng cấu trúc dữ liệu tương đương để thay thế kiểu danh sách liên kết.

Đề 10

Lập 1 chương trình trên ngôn ngữ lập trình tùy chọn C++/Java/C# gồm 2 tác vụ (Task1, Task2) hoạt động ở chế độ không đồng bộ như sau:

Task1:

1. Viết 1 hàm gồm 1 vòng lặp nhập liên tục các kí tự từ bàn phím, lưu các kí tự nhập vào một biến mảng A, và vòng lặp dừng lại khi nhập kí tự '#'. Viết hàm main() gọi hàm, dịch và chạy chương trình.
2. Thiết lập **Task1** bằng cách đặt hàm trên vào 1 luồng, kích hoạt luồng trong hàm main(). Dịch chương trình.

Task2:

3. Viết 1 hàm có 1 vòng lặp chỉ dừng lặp khi vòng lặp của Task1 dừng lại. Vòng lặp này luôn hiển thị các kí tự được lưu trong biến mảng A. Bổ sung lệnh gọi hàm trong hàm main(), dịch chương trình.
4. Thiết lập **Task2** bằng cách đặt hàm trên vào 1 luồng, thêm lệnh hiển thị số bộ nhớ vật lý (RAM) còn lại của hệ thống, kích hoạt luồng trong hàm main(). Dịch và chạy chương trình.

Chú ý: Có thể sử dụng cấu trúc dữ liệu tương đương để thay thế kiểu hàng đợi.

Đề 11

Lập 2 chương trình trên ngôn ngữ lập trình tùy chọn **C++/Java/C#** gồm CT1(client), CT2 (server) sử dụng Socket API, giao thức TCPIP truyền nhận dữ liệu tại IP="127.0.0.1" (localhost), port=5000 và hoạt động như sau:

CT1:

- Viết 1 hàm gồm 1 vòng lặp nhập vào từ bàn phím mảng số thực (nhập số phần tử **n**, sau đó nhập từng phần tử mảng), nếu nhập **n** thỏa $n \leq 0$ thì thoát khỏi vòng lặp. Viết hàm main() gọi hàm.
- Bổ sung lệnh truyền giá trị **n** và truyền biến mảng đến cho CT2 trong vòng lặp. Dịch chương trình CT1.

CT2:

- Viết 1 hàm có 1 vòng lặp vô hạn nhận giá trị biến nguyên **n** được gửi đến từ CT1, Nếu nhận được **n** thì:

Trường hợp 1: Nếu $n \leq 0$ thì thoát khỏi vòng lặp.

Trường hợp 2: Trái lại, nhận tiếp mảng **n** số thực được gửi đến từ CT1 và hiển thị trung bình cộng của **n** số nhận được. Viết hàm main() gọi hàm, dịch chương trình CT2.

Rồi lần lượt chạy CT2 và chạy CT1 để xem hiệu ứng nhập mảng ở CT1, hiển thị mảng ở CT2.

- Đặt hàm ở câu 3 vào 1 timer, bổ sung lệnh kích hoạt timer trong hàm main(). Dịch chương trình.

Biên dịch và chạy lần sau cùng:

+ Đóng tất cả CT1, CT2 đang chạy.

+ Chạy chương trình CT2 sau đó chạy CT1 để xem hiệu ứng kết nối, truyền và nhận dữ liệu.

Đề 12

Lập 1 chương trình trên ngôn ngữ lập trình tùy chọn **C++/Java/C#** gồm 3 tác vụ (Task1, Task2, Task3) hoạt động ở chế độ đồng bộ và không đồng bộ như sau:

Task1:

- Viết 1 hàm gồm 1 vòng lặp nhập liên tục các kí tự từ bàn phím, lưu các kí tự nhập vào một biến tổng thể **C** có cấu trúc dữ liệu kiểu bộ đệm vòng, và vòng lặp dừng lại khi nhập kí tự '!'. Viết hàm main() gọi hàm, dịch và chạy chương trình.
- Thiết lập Task1 bằng cách đặt hàm trên vào 1 luồng, kích hoạt luồng trong hàm main(). Dịch và chạy lại chương trình.

Task2:

- Viết 1 hàm có 1 vòng lặp chỉ dừng lặp khi vòng lặp của Task1 dừng lại. Vòng lặp này luôn hiển thị kí tự theo thứ tự nhập được lưu trong biến tổng thể **C** của Task1. Bổ sung lệnh gọi hàm trong hàm main(), dịch và chạy lại chương trình.
- Thiết lập Task2 bằng cách đặt hàm trên vào 1 luồng, kích hoạt luồng trong hàm main(). Dịch và chạy lại chương trình.

Task3:

- Viết 1 hàm đếm số kí tự bằng 'A' do Task1 tạo ra.

Task3 được gọi đồng bộ với Task1 nghĩa là Task3 chỉ thực sự hoạt động khi Task1 kết thúc. Gọi Task3 trong hàm main(). Dịch và chạy lại chương trình.

Chú ý: Có thể sử dụng cấu trúc dữ liệu tương đương để thay thế kiểu cấu trúc bộ đệm vòng.

Đề 13

Lập 2 chương trình khác nhau trên ngôn ngữ lập trình tùy chọn C++/Java/C# với yêu cầu sau:

CT1 (có hàm main):

1. Viết 1 hàm gồm 1 vòng lặp nhập 1 kí tự từ bàn phím, nếu kí tự là '#' thì thoát khỏi vòng lặp, trái lại thì ghi kí tự vào file vidu.txt (file text), viết hàm main() gọi hàm.
2. Đặt hàm vào 1 luồng, kích hoạt luồng trong hàm main(). Dịch và chạy chương trình CT1.

CT2 (có hàm main):

3. Viết 1 hàm có 1 vòng lặp vô hạn đọc kí tự ở file vidu.txt. Đọc được 1 kí tự thì đóng file và hiển thị mã hexa của kí tự. Nếu kí tự đọc được là '#' thì thoát vòng lặp. In ra số lượng kí tự bằng kí tự 'A'.

Viết hàm main() gọi hàm, dịch và chạy chương trình CT2.

4. Đặt hàm vào 1 luồng, kích hoạt luồng trong hàm main(). Dịch và chạy lại chương trình CT2. Chạy cả CT1 và CT2 đồng thời để thấy hiệu ứng trao đổi dữ liệu.

Đề 14

Lập 1 chương trình trên ngôn ngữ lập trình tùy chọn C++/Java/C# gồm 2 tác vụ (Task1, Task2) hoạt động ở chế độ không đồng bộ như sau:

Task1:

1. Viết 1 hàm gồm 1 vòng lặp nhập 1 xâu kí tự **st** từ bàn phím (**st** là một biến tổng thể). Nếu xâu **st** được nhập là “quit” thì thoát khỏi vòng lặp, trái lại ghi vào thành một dòng text của một file text là vidu.txt. Viết hàm main() gọi hàm, dịch chương trình.
2. Thiết lập Task1 bằng cách đặt hàm trên vào 1 timer hoặc 1 luồng, kích hoạt timer/luồng trong hàm main(). Dịch chương trình.

Task2:

3. Viết 1 hàm có 1 vòng lặp vô hạn đọc xâu kí tự ở file vidu.txt. Đọc được 1 dòng thì đóng file. Nếu dòng đọc được là “quit” thì thoát vòng lặp, trái lại hiển thị biến xâu kí tự. Hiển thị số lượng dòng text của file vidu.txt. Bổ sung lệnh gọi hàm trong hàm main.
4. Thiết lập Task2 bằng cách đặt hàm trên vào 1 timer hoặc 1 luồng, kích hoạt time/luồng trong hàm main(). Dịch và chạy lại chương trình.

Đề 15

Lập 1 chương trình trên ngôn ngữ lập trình tùy chọn C++/Java/C# gồm 2 timer (Timer1, Timer2), hoạt động như sau:

Timer1:

1. Viết 1 hàm gồm 1 vòng lặp nhập 1 kí tự từ bàn phím, lưu vào 1 biến tổng thể **c**, hiển thị mã hexa của kí tự. Nếu kí tự là '!' thì thoát khỏi vòng lặp. Viết hàm main() gọi hàm.
2. Đặt hàm vào 1 timer (Timer1) cứ 7ms (1ms=1/1000 giây) thực hiện 1 lần, kích hoạt Timer1 trong hàm main(). Dịch chương trình.

Timer2:

3. Viết 1 hàm có 1 vòng lặp hiển thị thời gian của hệ thống, nếu biến tổng thể **c** là '!' thì thoát vòng lặp (xem câu 1). Bổ sung lệnh gọi hàm trong hàm main(), dịch chương trình.
4. Đặt hàm vào 1 timer (Timer2) cứ 3ms thực hiện 1 lần, kích hoạt Timer2 trong hàm main(). Dịch và chạy chương trình.

Đề 16

Lập 2 chương trình khác nhau trên ngôn ngữ lập trình tùy chọn C++/Java/C# gồm CT1(client), CT2 (server) sử dụng Socket API, giao thức TCPIP truyền nhận dữ liệu tại IP="127.0.0.1" (localhost), port=5000 và hoạt động như sau:

CT1 (có hàm main):

1. Viết 1 hàm gồm 1 vòng lặp nhập vào từ bàn phím mảng số nguyên (nhập số phần tử **n**, sau đó nhập từng phần tử mảng), nếu nhập **n** thỏa $n \leq 0$ thì thoát khỏi vòng lặp. Viết hàm main() gọi hàm.
2. Bổ sung lệnh truyền giá trị **n** và truyền biến mảng theo thứ tự ngược đến cho CT2 trong vòng lặp (cần làm). Dịch chương trình CT1.

CT2 (có hàm main):

3. Viết 1 hàm có 1 vòng lặp vô hạn nhận giá trị biến nguyên **n** được gửi đến từ CT1, Nếu nhận được **n** thì:
Trường hợp 1: Nếu $n \leq 0$ thì thoát khỏi vòng lặp.
Trường hợp 2: Trái lại, nhận tiếp mảng **n** số nguyên được gửi đến từ CT1 và hiển thị giá trị của mảng theo thứ tự ngược sau khi nhận được. Viết hàm main(), dịch chương trình CT2.
Rồi lần lượt chạy CT2 và chạy CT1 để xem hiệu ứng nhập mảng ở CT1, hiển thị mảng ở CT2.
4. Đặt hàm ở câu 3 vào 1 thread, bổ sung lệnh kích hoạt luồng trong hàm main(). Dịch chương trình.

Biên dịch lần cuối:

- + Đóng tất cả CT1, CT2 đang chạy.
- + Chạy chương trình CT2 sau đó chạy CT1 để xem hiệu ứng kết nối, truyền và nhận dữ liệu.

Đề 17

Lập 1 chương trình trên ngôn ngữ lập trình tùy chọn **C++/Java/C#** gồm 2 tác vụ (Task1, Task2) hoạt động ở chế độ không đồng bộ như sau:

Task1:

1. Viết 1 hàm gồm 1 vòng lặp sinh số nguyên ngẫu nhiên 32bit cho đến khi gặp số chẵn cho 22092021 thì dừng vòng lặp. Viết hàm main() gọi hàm.
2. Thiết lập **Task1** bằng cách đặt hàm trên vào 1 timer chu kỳ 5 mili giây hoặc 1 luồng, kích hoạt timer/luồng trong hàm main(). Dịch chương trình.

Task2:

3. Viết 1 hàm có 1 vòng lặp chỉ dừng lặp khi vòng lặp của Task1 dừng lại. Vòng lặp này luôn hiển thị giá trị nhỏ nhất trong các số lớn hơn 2021 được **Task1** tạo ra tại mọi thời điểm chạy. Bổ sung lệnh gọi hàm trong hàm main().
4. Thiết lập **Task2** bằng cách đặt hàm trên vào 1 timer hoặc 1 luồng, kích hoạt timer/luồng trong hàm main(). Thêm lệnh phát ra tiếng beep khi hiển thị giá trị nhỏ nhất trên. Dịch và chạy chương trình.

Đề 18

Lập 2 chương trình khác nhau trên ngôn ngữ lập trình tùy chọn **C++/Java/C#** gồm CT1(client), CT2 (server) sử dụng Socket API, giao thức TCP/IP truyền nhận dữ liệu tại IP="127.0.0.1" (localhost), port=5000 và hoạt động như sau:

CT1 (có hàm main):

1. Viết 1 hàm gồm 1 vòng lặp nhập vào từ bàn phím mảng số nguyên (nhập số phần tử $n \geq 3$, sau đó nhập từng phần tử mảng), nếu nhập n thỏa $n < 3$ thì thoát khỏi vòng lặp. Viết hàm main() gọi hàm.
2. Bổ sung lệnh truyền giá trị n và truyền biến mảng đến cho CT2 trong vòng lặp. Dịch chương trình CT1 (nhưng không chạy).

CT2 (có hàm main):

3. Viết 1 hàm có 1 vòng lặp vô hạn nhận giá trị biến nguyên n được gửi đến từ CT1, Nếu nhận được n thì:
Trường hợp 1: Nếu $n < 3$ thì thoát khỏi vòng lặp.
Trường hợp 2: Trái lại, nhận tiếp mảng n số nguyên được gửi đến từ CT1 và hiển thị 3 giá trị đầu tiên xếp theo thứ tự từ thấp đến cao của mảng sau khi nhận được (3 giá trị đầu tiên có thể luôn thay đổi khi CT1 đang chạy). Viết hàm main(), dịch chương trình CT2.
Rồi lần lượt chạy CT2 và chạy CT1 để xem hiệu ứng nhập mảng ở CT1, hiển thị mảng ở CT2.
4. Đặt hàm ở câu 3 vào 1 timer, bổ sung lệnh kích hoạt timer trong hàm main(). Dịch chương trình.

Biên dịch và chạy lần sau cùng:

- + Đóng tất cả CT1, CT2 đang chạy.
- + Chạy chương trình CT2 sau đó chạy CT1 để xem hiệu ứng kết nối, truyền và nhận dữ liệu.

Đề 19

Lập 2 chương trình khác nhau trên ngôn ngữ lập trình tùy chọn **C++/Java/C#** gồm **CT1**(client), **CT2** (server) sử dụng Socket API, giao thức TCPIP truyền nhận dữ liệu tại IP="127.0.0.1" (localhost), port=5000 và hoạt động như sau:

CT1 (có hàm main):

1. Viết 1 hàm gồm 1 vòng lặp tạo ra số nguyên ngẫu nhiên **k**, vòng lặp dừng khi số nguyên sinh ra lớn hơn 1042021. Viết hàm main() gọi hàm, dịch chương trình.
2. Bổ sung lệnh truyền giá trị số nguyên sinh ra cho **CT2** trong vòng lặp. Dịch chương trình **CT1**.

CT2 (có hàm main):

3. Viết 1 hàm có 1 vòng lặp vô hạn nhận giá trị biến nguyên được gửi đến từ CT1. Kiểm tra, nếu **CT2** nhận được giá trị nguyên **k** thì:

Trường hợp 1: Nếu **k** > 1042021 thì thoát khỏi vòng lặp.

Trường hợp 2: Trái lại, hiển thị mã hexa giá trị lớn nhất được CT1 sinh ra tính tại thời điểm đang chạy (giá trị số lớn nhất có thể luôn thay đổi khi CT1 đang chạy). Viết hàm main(), dịch chương trình **CT2**.

Rồi lần lượt chạy **CT2** và chạy **CT1** để xem hiệu ứng nhập mảng ở **CT1**, hiển thị mảng ở **CT2**.

4. Đặt hàm ở câu 3 vào 1 timer, bổ sung lệnh kích hoạt timer trong hàm main(). Dịch chương trình.

Biên dịch và chạy lần sau cùng:

+ Đóng tất cả **CT1**, **CT2** đang chạy.

+ Chạy chương trình **CT2** sau đó chạy **CT1** để xem hiệu ứng kết nối, truyền và nhận dữ liệu.

Đề 20

Lập 1 chương trình trên ngôn ngữ lập trình tùy chọn **C++/Java/C#** gồm 2 tác vụ (Task1, Task2) hoạt động ở chế độ không đồng bộ như sau:

Task1:

1. Viết 1 hàm gồm 1 vòng lặp nhập liên tục các kí tự từ bàn phím, lưu các kí tự nhập vào một biến cấu trúc **Q** kiểu hàng đợi, và vòng lặp dừng lại khi nhập kí tự '#'. Viết hàm main() gọi hàm, dịch và chạy chương trình.
2. Thiết lập **Task1** bằng cách đặt hàm trên vào 1 luồng, kích hoạt luồng trong hàm main(). Dịch và chạy lại chương trình.

Task2:

3. Viết 1 hàm có 1 vòng lặp chỉ dừng lặp khi vòng lặp của Task1 dừng lại. Vòng lặp này luôn hiển thị các kí tự được lưu trong biến hàng đợi **Q**. Bổ sung lệnh gọi hàm trong hàm main(), dịch và chạy lại chương trình.
4. Thiết lập **Task2** bằng cách đặt hàm trên vào 1 luồng, kích hoạt luồng trong hàm main(). Dịch và chạy chương trình.

Chú ý: Có thể sử dụng cấu trúc dữ liệu tương đương để thay thế kiểu hàng đợi.